This code tries to match and replace our input field, replacing the underscore (_) character with a dash (-). We can test the analyzer with the following code.

Listing 7.42 Testing a custom pattern replace character filter

```
POST index_with_pattern_replace_filter/_analyze
{
  "text": "Apple_Boy_Cat",
  "analyzer": "my_pattern_replace_analyzer"
}
```

The output is "Apple-Boy-Cat" with all the underscores replaced with dashes.

The input text has been cleaned of unwanted characters, but there remains the job of splitting the text into individual tokens based on delimiters, patterns, and other criteria. That job is undertaken by a tokenizer component, discussed in the next section.

7.7 Tokenizers

The job of a tokenizer is to create tokens based on certain criteria. Tokenizers split incoming input fields into tokens that are usually the individual words of a sentence. Elasticsearch provides over a dozen tokenizers, each of which tokenizes fields based on the tokenizer's definition.

> **NOTE** As you can imagine, discussing all the tokenizers in this book would be impractical (and also boring to read). I have chosen a few important and popular tokenizers here so you can understand the concept and mechanics. Example code is available with the book's files.

7.7.1 The standard tokenizer

A `standard` tokenizer splits text into tokens based on word boundaries (whitespace delimiters) as well as punctuation (commas, hyphens, colons, semicolons, etc.). The following code uses the `_analyze` API to execute the tokenizer on a field:

```
POST _analyze
{
  "text": "Hello,cruel world!",
  "tokenizer": "standard"
}
```

This results in three tokens: "Hello", "cruel", and "world". The comma and the whitespace act as delimiters to tokenize the field into individual tokens.

The `standard` analyzer has only one attribute that can be customized: `max_token_length`. This attribute produces tokens of the size it defines (the default size is 255). We can set this property by creating a custom analyzer with a custom tokenizer.